CSCI-C 212 Exam 2 Fall 2026 (80 points)
Nov 2, 2026

PLEASE READ ALL DIRECTIONS BEFORE STARTING YOUR EXAM. DO NOT OPEN UNTIL YOU ARE TOLD TO DO SO.

Permitted Materials: This is a closed-note exam except for your aid sheet. No electronic devices or communication with anyone other than proctors and the professor is allowed. *Cheating or talking with others will result in an automatic F in the course.*

Imports and Design Recipe: Unless a question states otherwise, you may use any Java class without writing its corresponding `import`. You do not need to write the full design recipe (documentation, tests), though you may do so if it helps.

Errors on Exam: If you believe you have found an error on the exam, raise your hand and notify a proctor.

Upon Finishing: When you are finished, check over your work. If more than 10 minutes remain, submit your exam and aid sheet to a proctor, show your ID, then quietly exit. No one may leave during the final 10 minutes.

Removing Sheets of Paper: Do not tear out, detach, or remove any pages from this exam. If you use the scratch sheet of paper, please label the question on the sheet and put a note on the question.

Time Limit: You have **75 minutes** to complete the exam.

Good luck!

Question	Points	Score
1	20	
2	60	
Total:	80	

PUT YOUR NAME ON THE VERY LAST PAGE.

1. (20 points) Design the `Map<String, List<Integer>> kLowestGrades(Map<String, List<Integer>> grades, int k)` method that, when given a map of names to numeric grades *grades* and an integer $k \geq 1$, returns a lexicographically-ordered map of the lowest k grades, if they exist, from each of the names. Each list should be sorted in increasing order.

   For example, suppose we define *grades* as follows:

   $$\begin{aligned}
   \texttt{"Ransom"} &\mapsto [70, 45] \\
   \texttt{"Sunny"} &\mapsto [101, 22, 22, 23, 84] \\
   \texttt{"Matthew"} &\mapsto [98] \\
   \texttt{"Peter"} &\mapsto [] \\
   \texttt{"Elliot"} &\mapsto [100, 101, 45] \\
   \texttt{"Carley"} &\mapsto [67, 82, 103] \\
   \texttt{"Laura"} &\mapsto [100, 95]
   \end{aligned}$$

   If $k = 2$, then `kLowestGrades` returns:

   $$\begin{aligned}
   \texttt{"Carley"} &\mapsto [67, 82] \\
   \texttt{"Elliot"} &\mapsto [45, 100] \\
   \texttt{"Laura"} &\mapsto [95, 100] \\
   \texttt{"Matthew"} &\mapsto [98] \\
   \texttt{"Peter"} &\mapsto [] \\
   \texttt{"Ransom"} &\mapsto [45, 70] \\
   \texttt{"Sunny"} &\mapsto [22, 22]
   \end{aligned}$$

   Huge *hint: use a* `PriorityQueue!`

THE SKELETON CODE IS ON THE NEXT PAGE.
WRITE YOUR SOLUTION THERE.

```
/**
 * Returns a lexicographically ordered map from each name to its lowest k grades.
 * Each returned list is sorted in increasing order.
 * @param grades map from names to their numeric grades.
 * @param k maximum number of lowest grades to keep for each name.
 * @return map from each name to a sorted list of its lowest k grades.
 */
static Map<String, List<Integer>> kLowestGrades(Map<String, List<Integer>> grades,
                                                int k) {



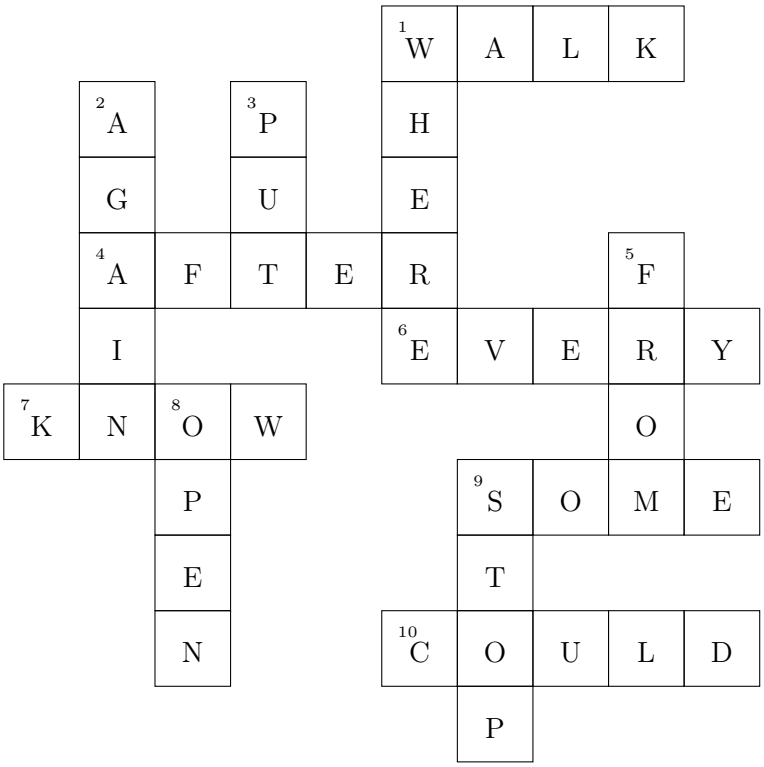






}
```

2. Consider a crossword puzzle defined as a `String[][]`, where each cell is either a *one-string*, i.e., a string of one character, or `null` to indicate that the cell is not used by the crossword.



The same filled crossword can be represented as a `String[][]`:

```
String[][] crossword = {
    {null, null, null, null, null, "W",  "A",  "L",  "K",  null},
    {null, "A",  null, "P",  null, "H",  null, null, null, null},
    {null, "G",  null, "U",  null, "E",  null, null, null, null},
    {null, "A",  "F",  "T",  "E",  "R",  null, null, "F",  null},
    {null, "I",  null, null, null, "E",  "V",  "E",  "R",  "Y" },
    {"K",  "N",  "O",  "W",  null, null, null, null, "O",  null},
    {null, null, "P",  null, null, null, "S",  "O",  "M",  "E" },
    {null, null, "E",  null, null, null, "T",  null, null, null},
    {null, null, "N",  null, null, "C",  "O",  "U",  "L",  "D" },
    {null, null, null, null, null, null, "P",  null, null, null}
};
```

In this question, you will design a `Crossword` class to represent crossword puzzles and search for words in the puzzle.

(Fun fact: this question is inspired by my AP Computer Science A exam from 2016. Yes, that exam was from hell... I hope this one is not!)

CONTINUE ONTO THE NEXT PAGE.

(a) (6 points) Design the `Crossword` class, whose constructor receives the `String[][]` board and copies it into a *referentially final* instance variable array. **Do not just reassign the reference!**

```
class Crossword {

}
```

(b) (4 points) Certain cells in a crossword puzzle are *labeled*, meaning they start at least one word. Cells that are *not* labeled cannot start a word. There are two rules for whether a cell is labeled:

1. The cell is in bounds and non-`null`.
2. The cell starts either an "across word" or a "down word."

Design the `boolean isLabeled(int r, int c)` method that returns whether the cell at (r, c) is labeled according to the above rules.

We provide two helper methods: `boolean startsAcross(int r, int c)` and `boolean startsDown(int r, int c)` that determine if the cell at (r, c) starts a word going across or going down, respectively.

We also provide the helper method `boolean inBounds(int r, int c)`, which returns whether the cell (r, c) is within the bounds of the crossword.

Your implementation must use these three methods correctly or you will not receive credit.

THE SKELETON CODE IS ON THE NEXT PAGE.
WRITE YOUR SOLUTION THERE.

```
class Crossword {

  /* Other code omitted. */

  /**
   * Determines whether the cell at (r, c) starts an across word.
   * @param r row index of the cell.
   * @param c column index of the cell.
   * @return true if the cell starts an across word; false otherwise.
   */
  boolean startsAcross(int r, int c) { /* Implementation omitted. */ }

  /**
   * Determine whether the cell at (r, c) starts a down word.
   * @param r row index of the cell.
   * @param c column index of the cell.
   * @return true if the cell starts a down word; false otherwise.
   */
  boolean startsDown(int r, int c) { /* Implementation omitted. */ }

  /**
   * Determines whether the cell at (r, c) is in bounds. Note that
   * a cell that is not used by the board can still be in bounds.
   * @param r row index of the cell.
   * @param c column index of the cell.
   * @return true if the cell is in bounds; false otherwise.
   */
  boolean inBounds(int r, int c) { /* Implementation omitted. */ }

  /**
   * Determines whether the cell at (r, c) is to be labeled.
   * @param r row index of the cell.
   * @param c column index of the cell.
   * @return true if the cell is to be labeled; false otherwise.
   */
  boolean isLabeled(int r, int c) {









  }
}
```

(c) (20 points) Design the `List<String> findWords(int r, int c)` method that finds the words when starting at the labeled cell (r, c). Note that intersecting into another labeled word does not start a new word. Below is the pseudocode to write this algorithm, which extends out the current string across and down as long as the cells exist.

To exemplify the algorithm, calling `findWords(0, 5)` on the above crossword returns `["WALK", "WHERE"]`. Importantly, the returned list does *not* include `"EVERY"`.

Your solution must use the three helper methods provided in part (b) correctly or you will not receive credit.

```
findWords(r, c):
  words <- []
  if startsAcross(r, c) then
    c' <- c
    str <- ""
    while (r, c') is in bounds and crossword at (r, c') is not null do
      str <- str + crossword at (r, c')
      increment c'
    add str to words

  if startsDown(r, c) then
    r' <- r
    str <- ""
    while (r', c) is in bounds and crossword at (r', c) is not null do
      str <- str + crossword at (r', c)
      increment r'
    add str to words

  return words
```

THE SKELETON CODE IS ON THE NEXT PAGE.
WRITE YOUR SOLUTION THERE.

```
class Crossword {

  /* Other code omitted. */

  /**
   * Finds the words that start at the labeled cell at (r, c).
   * @param r row index of the labeled cell.
   * @param c column index of the labeled cell.
   * @return list of either 1 or 2 words.
   */
  List<String> findWords(int r, int c) {









  }
}
```

CONTINUE ONTO THE NEXT PAGE.

(d) (14 points) Overload the `findWords` method to return *all* of the words in the crossword. **Your solution must use `isLabeled` correctly or you will not receive credit.**

```
class Crossword {

  /* Other code omitted. */

  /**
   * Finds all of the words in the crossword puzzle from labeled cells.
   * @return list with all words.
   */
  List<String> findWords() {









  }
}
```

(e) (8 points) Design the `List<String> findWordsOfLength(int n)` method that returns all words that are exactly of length n. Part of the code has been provided. **You must use only the Stream API.**

```
class Crossword {

  /**
   * Finds only the words that are of length n.
   * @param n length of words to find.
   * @return list of words of length n.
   */
  List<String> findWordsOfLength(int n) {
    return this.findWords().stream()









  }
}
```

(f) (8 points) Override the `public boolean equals(Object o)` method to return whether two `Crossword` objects are equal. They are equal if and only if their underlying arrays are equal.

```
class Crossword {

  /* Other code omitted. */

  @Override
  public boolean equals(Object o) {
    if (_____________________________________________________) {
      ________________________________________;
    } else {









    }
  }
}
```

Name: _____

IU Email: _____

Scratch work